

Software Design Document for Scholar Path

Version 1.4.0 approved

**Prepared by Charlie Kaing, Joshua Hanscom, Ahin Huq, Jonathan Little,
Abigail Diaz**

Cal State LA

11/29/24

Table of Contents.....	<pg 2>
Revision History.....	<pg 4>
1. Introduction.....	<pg 5>
1.1. Purpose.....	<pg 5>
1.2. Document Conventions.....	<pg 5>
1.3. Intended Audience and Reading Suggestions.....	<pg 5>
1.4. System Overview.....	<pg 6>
Design Considerations.....	<pg 7>
2.1. Assumptions and dependencies.....	<pg 7>
2.2. General Constraints.....	<pg 7>
2.3. Goals and Guidelines.....	<pg 7>
2.4. Development Methods.....	<pg 8>
Architectural Strategies.....	<pg 10>
System Architecture.....	<pg 12>
4.1. Logical View.....	<pg 12>
4.2. Development View.....	<pg 14>
4.3. Process View.....	<pg 17>
4.4. Physical View.....	<pg 17>
Policies and Tactics.....	<pg 18>
5.1. Specific Products Used.....	<pg 18>
5.2. Requirements traceability.....	<pg 18>
5.3. Testing the software.....	<pg 19>
5.4. Engineering trade-offs.....	<pg 19>
5.5. Guidelines and conventions.....	<pg 19>
5.6. Protocols.....	<pg 19>
5.7. Maintaining the software.....	<pg 20>
5.8. Interfaces.....	<pg 20>
5.9. System's deliverables.....	<pg 20>
5.10. Abstraction.....	<pg 21>
Detailed System Design.....	<pg 22>
6.x Name of Module.....	<pg 22>
6.x.1 Responsibilities.....	<pg 22>
6.x.2 Constraints.....	<pg 22>
6.x.3 Composition.....	<pg 22>
6.x.4 Uses/Interactions.....	<pg 22>
6.x.5 Resources.....	<pg 22>
6.x.6 Interface/Exports.....	<pg 22>
Detailed Lower level Component Design.....	<pg 25>
7.x Name of Class or File.....	<pg 25>
7.x.1 Classification.....	<pg 25>
7.x.2 Processing Narrative(PSPEC).....	<pg 25>
7.x.3 Interface Description.....	<pg 25>
7.x.4 Processing Detail.....	<pg 25>
7.x.4.1 Design Class Hierarchy.....	<pg 25>
7.x.4.2 Restrictions/Limitations.....	<pg 25>
7.x.4.3 Performance Issues.....	<pg 25>

7.x.4.4	Design Constraints.....	<pg 25>
7.x.4.5	Processing Detail For Each Operation.....	<pg 25>
	Database Design.....	<pg 26>
	User Interface.....	<pg 29>
9.1	Overview of User Interface.....	<pg 29>
9.2	UX Standards.....	<pg 29>
9.3	Screen Frameworks or Images.....	<pg 29>
9.4	User Interface Flow Model.....	<pg 29>
	Database Design	
	Requirements Validation and Verification.....	<pg 30>
	Glossary.....	<pg 31>
	References.....	<pg 32>

Revision History(Charlie Kaing)

Name	Date	Reason For Changes	Version
Version 1.0.0	10/7	Basic Map using Navigator API	1.0.0
Version 1.1.0	10/15	User Markers, Amenity Finder	1.1.0
Version 1.2.0	10/20	Study group feature, SQL Integration, CIN Verification	1.2.0
Version 1.3.0	11/1	Destination selection and rerouting, SQL Database, PostgreSQL	1.3.0
Version 1.4.0	11/10	ScholarPath Server Database Integration to Frontend, API development	1.4.0

<Add rows as necessary when the document is revised. This document should be consistently updated and maintained throughout your project. If ANY requirements are changed, added, removed, etc., immediately revise your document.>

1. Introduction_(AD)

1.1 Purpose

This document specifies the software requirements for Scholar Path, a web-based campus navigation system. Scholar Path aims to help students navigate the university's campus, locate classrooms, resources and amenities. The scope of the outlined software requirements apply to the initial release of the application, including real time navigation, map interactions, search functionalities, and directed paths to locations.

1.2 Document Conventions

The following conventions have been applied:

- **Bold** text is used to highlight major titles, people involved or features.
- Courier New font is used to refer to samples of code, file names, or system commands.
- *Italics* to refer to Scholar Path.

Priorities

Higher-level requirements inherit their priorities from how important they are to the base structure of the program, with priority given to the most fundamental requirements. When it comes to each detailed requirement that extends on higher level requirements, the same approach would be given by allocating priority based on the requirements' necessity to the higher-level requirement achievement.

1.3 Intended Audience and Reading Suggestions

This SRS is intended for the following audiences:

- **Developers:** To have a clear understanding of the system's functionality and technical constraints.
- **Project Managers:** To ensure that project goals align with client requirements.
- **Product Owner:** To manage the product backlog and redefine the requirements.
- **Scrum Master:** To ensure the requirements are clear and understood.
- **Testers:** To develop test cases that verify the program performs as specified.
- **End Users:** To understand the functionality of the program.

Focus

The document is structured with an overview of the system in mind and design/architectural considerations first, followed by detailed technical requirements.

Developers and testers may focus on sections 3 and 4 for implementation details managers and marketing staff may find sections 1 and 2 most relevant.

1.4 System Overview

The Scholar Path application is designed to help students easily navigate through the university campus.

Services Provided

- An Interactive campus map with real time GPS location.
- Search functionality for buildings, classrooms, and amenities (cafeteria, libraries)
- Route recommendations based on selected destinations
- Creation of personal account to save classes
- Group meetings locator

Design Focus

The program is designed to be a web-based application that provides interactive navigation and works on browsers on both desktop and mobile devices.

2. Design Considerations

This section describes many of the issues which need to be addressed or resolved before attempting to devise a complete design solution.

2.1 Assumptions and Dependencies (Charlie Kaing)

Scholar Path is a web mobile app to ensure compatibility with majority of devices and provide ease of development the following assumptions are made:

- Users will be using either a mobile device on Android or IOS with a gps or desktop with more limited functionality
- The user will be using the app on campus at Cal State LA to locate classes and schedules

Scholar Path uses the following API as a dependences:

- Navigator API for gps location services
- Leaflet for Interactive map handling

2.2 General Constraints(charlie Kaing)

The Web app will only be able to be used on Campus at Cal State LA and will be used to locate classrooms. The web app currently is unusable outside of campus outside of viewing of classrooms.

- Additional location information such as events and food are currently not being planned
- Database will use the schedule for fall 2024
- Users will be using a phone that is compatible with the navigator and leaflet api.

2.3 Goals and Guidelines (Joshua Hanscom)

Describe any goals, guidelines, principles, or priorities which dominate or embody the design of the system's software. For each such goal or guideline, unless it is implicitly obvious, describe the reason for its desirability. Feel free to state and describe each goal in its own subsection if you wish. Such goals might be:

- Commit version changes specific to functionality worked on
 - Avoids redundancy of file versions and makes each file relevant to their last update via commit

- Commits should be made from a branch when possible
 - Integration errors are handled before they can hinder the work of other members
- Commit messages should start with the committer(s) name(s) (e.g. Josh - ...)
 - Allows easy communication to understand changes
 - Keeps record of member contributions
- Merge requests should be reviewed/confirmed by more than one member
 - Avoids implementation of generally disapproved coding methods and functionality
- Members should commit a refactored version of functionality when possible
 - Refactored code is particularly important
- Deliver 9 functional features by the end of PI 24.2
- The final product should be presented in a way that is intuitive to the average user (CSULA Student)

2.4 Development Methods (Joshua Hanscom)

Briefly describe the method or approach used for this software design. If one or more formal/published methods were adopted or adapted, then include a reference to a more detailed description of these methods. If several methods were seriously considered, then each such method should be mentioned, along with a brief explanation of why all or part of it was used or not used.

These would be things such as the ‘Water Fall Development’ methods, ‘Agile Development’, ‘Unplanned Mad Scramble Development’, or other development models and variations. Describe how these were applied in the case of your project.

Agile Development: Ensures that the goals to reach the end product are modified accordingly to fit all potentially unforeseen circumstances. These include but are not limited to learning Agile, Scrum, and Change Management; responding to conflicting member schedules, improving communication for daily product development progress.

- Scrum Framework: Works around our relatively small development team to ensure that the tasks that we, as a team, plan reflect the requirements and validity of what is required to deliver our product.
 - 3 Planning Increments: PI 24.0 served as an introduction into foundation software engineering concepts and Agile development methods. PI 24.1 has allowed the team to fill our product backlog, begin prioritization of said backlog, and begin development of our product via the already completed sprints. With the rest of PI 24.1 and all of PI 24.2, we as a team will continue developing the requirements of the product, planning, reviewing, and prioritizing the tasks needed to fulfill our products requirements.
 - 3, 6, 6 Sprints accordingly
 - Sprint Planning - Friday 10am: Ensuring our team backlog is prioritized for the week-long sprint. Features to be implemented

are classified by their connection to three pages: Login, Map, and Profile. When necessary, Scrum roles within the team are rotated.

- Sprint Review - Saturday 6pm: As a team, review the tasks completed, noting their relation to the team backlog. Additionally, consider the performance of team members in Scrum roles.
- Sprint Retrospective - Saturday 6pm: Considering the main outcomes discussed in the sprint review, the team discusses changes that could have been made to prevent issues that occurred and efficiencies that could have been achieved.
- Backlog Refinement - Friday 8pm: With the progress achieved during the sprint, the acting Product Manager defines new requests based on the product and the existing roadmap. The new backlog is then reviewed by the team (Saturday 5pm).

3. Architectural Strategies (Ahin Huq)

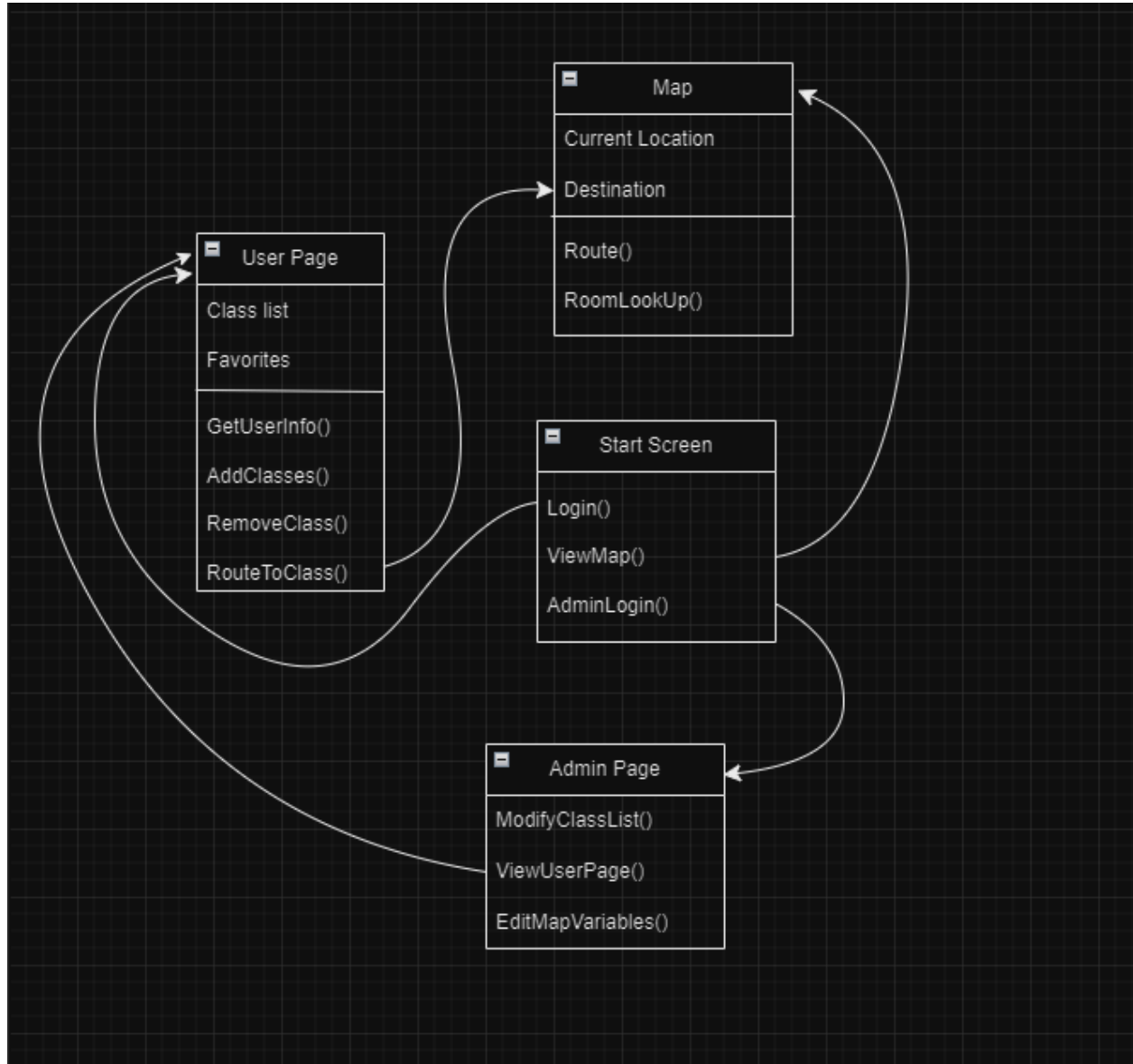
Describe any design decisions and/or strategies that affect the overall organization of the system and its higher-level structures. These strategies should provide insight into the key abstractions and mechanisms used in the system architecture. Describe the reasoning employed for each decision and/or strategy (possibly referring to previously stated design goals and principles) and how any design goals or priorities were balanced or traded-off. Such decisions might concern (but are not limited to) things like the following:

- Model View Controller (MVC): Project follows MVC to separate concerns and improve maintainability.
- A Product of a Specific Type: for its backend, Scholar Path is using JavaScript; with MySQL as the database; and it's JavaScript (using the Leaflet API) for map rendering.
- [J]Reinventing the wheel: We use Navigator API for GPS and Leaflet API for map functionality pageIndex to avoid rewriting parts of previously written software.
- Next steps to improve/extend the software: Indoor navigation and impart real-time event tracking on campus.
- At a high level, the user interface paradigms are as follows: Responsiveness- The UI is designed responsively and adapts to both mobile and desktop browsers.
- Interfacing paradigms with hardware and/or software: The system is interfaced directly to the GPS hardware, but through its REST API, it connects to the MySQL database.
- Error Notification and Recovery: For example, lacks GPS signal or errors fetching maps will use user-friendly messages and retry to update maps automatically in case of failure.
- Memory management policy: The use of cache for map tiles for minimizing memory utilization, and non-necessary data off-loading to the server.
- Databases and external data storage: one database stores the locations of classrooms as well as user preferences calling SQL queries to access them.
- Data or control distribution on a network: The user data and map request is processed through HTTP requests to a centralized server.
- Control in general: We have controller classes in the backend to deal with user input and to manage interaction mapping.

- Concurrent execution and synchronization: Java's threading model comes in handy for allowing GPS and map updates to run concurrently without blocking the user interface.
- Methods of communication: data flow between client and server can be either through AJAX or RESTful APIs depending on the requirement.
- Other resource management: Server resources, like CPU and memory, are optimized using load balancing and efficient query management.

4. System Architecture

4.1 Logical View(Jon Little) ER Diagram



The ER diagram for Scholar Path captures the relationships between the various entities that manage the campus navigation system's data and page navigation. Here's a high level list of the entities: User Page, Map, Start Screen and Admin Page.

Architectural Components and Flow Layers in the MVC Framework Model

The goal is to manage the application's data logic and rules.

Interacts with the MySQL database for:

- Classroom locations.
- User preferences and profiles.
- Amenity data.

GPS location data is handled with the Navigator API.

View:

- The UI layer creates user interfaces using JavaScript.

It Includes:

- Login/Guest access page.
- Campus map page (using the Leaflet API).
- Profile management and settings page.
- Amenity and classroom navigation interfaces.
- Displays error notifications for login attempts.

The Controller does the following:

- Manages user inputs (like in a search bar)
- Backend logic implemented in JavaScript and Server implemented with JSON:
 - Handles user actions (login & map interactions).
 - RESTful API calls to interact with the model and database.
 - Controls GPS updates and map display.

High-Level Components

Frontend:

- Built with responsive JavaScript UI.
- Utilizes the Leaflet API for map display.
- Uses Graphhopper API for pedestrian routing.
- Uses Navigator API for the GPS interface.

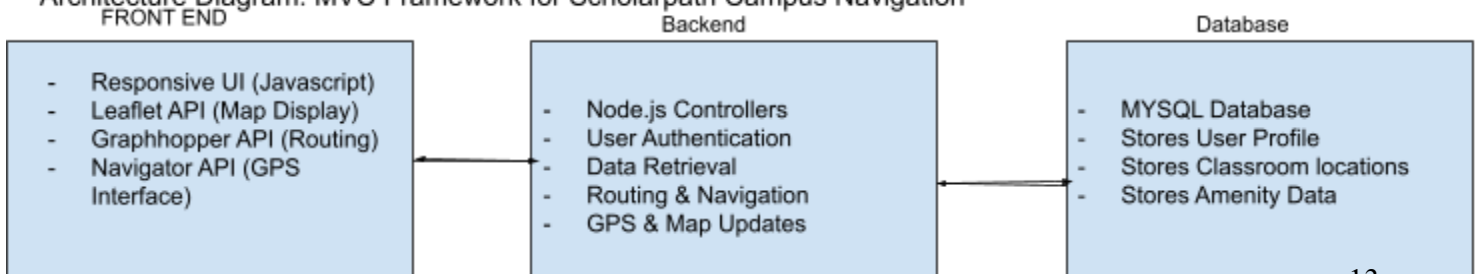
Backend:

- Node.js
- Controllers manage application logic
 - User authentication (login/guest access).
 - Data retrieval (classrooms).
 - Routing for navigation.
 - GPS and map updates.

Database:

- MySQL database:
- Stores user data and classroom locations, as well as amenity information.

Architecture Diagram: MVC Framework for Scholarpath Campus Navigation



4.2 Development View

4.2.1 Overview of Responsibilities

At a high level, the system's responsibilities include:

- User Management: Account creation, login, and guest access.
- Campus Map Services: Real-time location tracking, navigation, and interaction with campus amenities.
- Classroom and Study Group Tracking: Directions to classrooms and study group locations, and access to class schedules.
- Data Management: Storage and retrieval of user, campus and classroom information.

4.2.2 Subsystems and Components

Scholar Path is decomposed into the following top-level subsystems, each aligned with specific functional requirements:

4.2.2.1 User Management Subsystem

Role: Handles authentication, authorization, and user account functionalities, including guest access.

Responsibilities:

Account Creation (4.1.1.1): Allow users to create an account using an email and password.

User Login (4.1.1.2): Authenticate users and provide personalized access.

Guest Access (4.1.1.3): Allow temporary access with limited functionality.

4.2.2.2 Map and Navigation Subsystem

Role: Provides campus map services for location tracking, navigation, and amenity interactions.

Responsibilities:

Real-Time Location (4.1.2.1): Display user location as a map marker and update dynamically.

Map Navigation (4.1.2.2): Allow users to zoom, scroll, and pan across the map seamlessly.

Amenity Information (4.1.2.3): Provide clickable icons and pop-up details for campus amenities.

4.2.2.3 Classroom and Study Group Navigation Subsystem

Role: Facilitates classroom and study group location and navigation.

Responsibilities:

Classroom Directions (4.1.6): Store class schedules and provide navigation to classrooms.

Study Group Locations (4.1.7): Enable users to locate and navigate to study group locations.

4.2.2.4 Data Management Subsystem

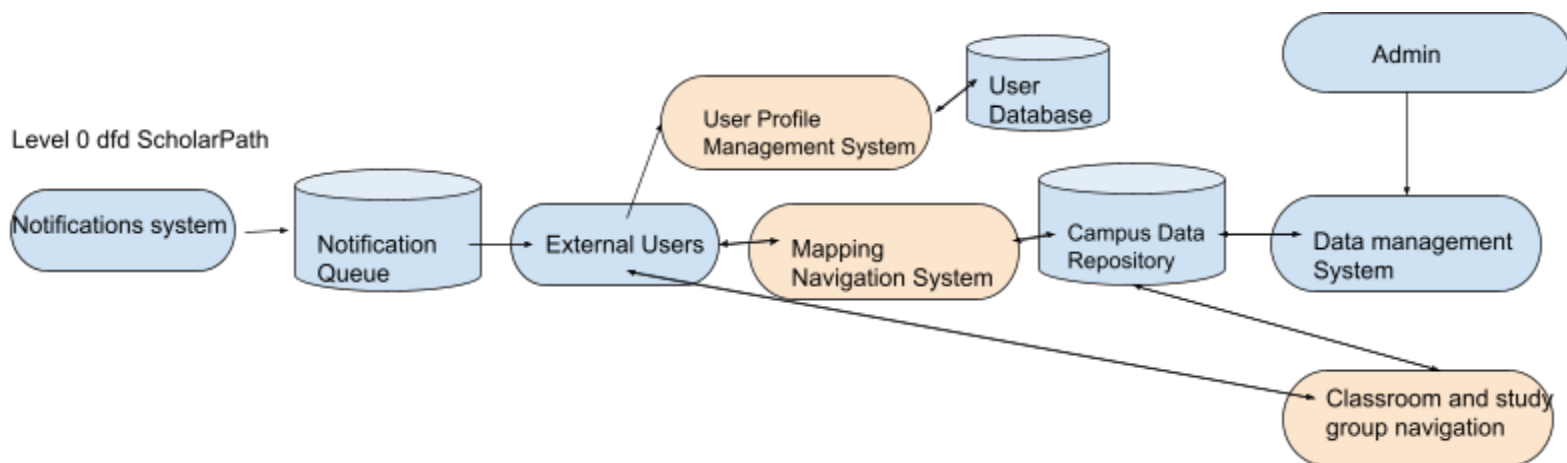
Role: Manages storage and retrieval of system data.

Responsibilities: Store user profiles, classroom locations, and amenity details.

4.2.2.5 Notification Subsystem

Role: Notifies of updates and notifications to users in real time.

Responsibilities: Notify users about system events, changes, or upcoming study groups (4.1.7.2.2).



The Level 0 DFD above shows how data flows between external entities (users, administrators), major subsystems, and data stores:

Entities:

- External Users:
 - Students
 - Administrators
- System Subsystems:
 - User Management
 - Mapping and Navigation
 - Classroom and Study Group Navigation
 - Data Management
 - Notification
 - Administration
- Data Stores:
 - User Database
 - Campus Data Repository
 - Notification Queue

4.3 Process View

Context Diagram

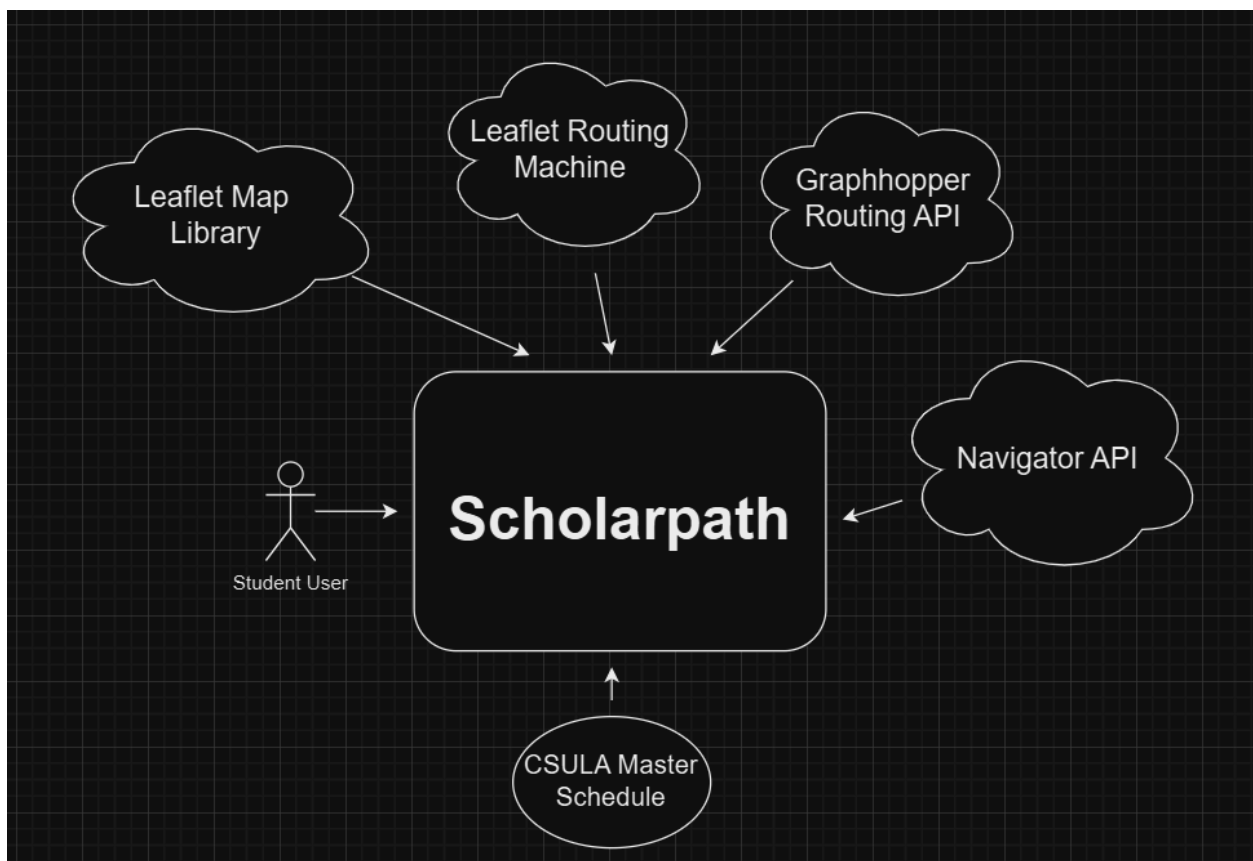
The Context Diagram below illustrates the relationships between Scholarpath and its external systems.

User Devices: Students, staff, and visitors access the application via web browsers.

Campus Data Systems: Real-time data is retrieved from systems like facilities management and scheduling from the CSULA Master Schedule.

External APIs: Utilizes Leaflet API for map rendering and Navigator API for geolocation.

The student will interact directly with the product. Our product, including front-end and back-end functionality, will interact with the external integrations and data to ultimately deliver a website that solves the original issue in a digestible way to the user.



5. Policies and Tactics

5.1 Choice of which specific products used (Ahin Huq)

IDE: Visual Studio Code(Chosen for wide range of extensions and the easy usability)

Front end: HTML5/JavaScript Back end integration : Java

Database: MySQL(for user data, build data, classroom locations)

API Library:

Navigator api for locating the user according to its current location on campus using gps location services.

Leaflet —> For an equivocal Dumpster Fire View Leaflet API to better handle this interactive map display.

Version control (GitHub used for: project source code / collaboration)

5.2 Plans for ensuring requirements traceability

Traceability of the requirements(Profile based Requirements traceability conformance)

In Case If You Want To Follow Any Other Method To Complete All Needed Of This Maybe You Have:

Requirement Mapping —Maps each functional / non-functional requirement to a module or task in the Development Backlog.

Jira/Backlog Tool : We usually use JIRA or similar kind of a tool to track the development process where all requirements listed in SRS are functional and can be traced through user stories or Epics.

Test Cases:Associated with each requirement are test cases to make sure it works as expected and also tested correctly from development to testing phase(percent traceability).

5.3 Plans for testing the software

Scholar Path- Testing plans

Unit Testing: Manual and automation tests are used to test individual components (e.g., GPS location services, map display) framework code. For testing the web app both the html5/javascript and Junit are used.

Integration Testing: In this, we will perform the integration of the map with database retrieval and GPS to ensure its working.

End-to-End Testing: End-to-end tests will check that a user can sign in, see the map and get any expected navigation output all the way through. Testing will be done in both desktop and mobile environments (CS3337_SWE_Software_TestPlan_Template).

Cal State LA Campus-specific Manual On-Campus Testing . Since the app was created for a university, we will hand test our encapsulated GPS functionality on campus to ascertain accuracy.

5.# Engineering trade-offs

One of the important tradeoffs we made was between real-time GPS accuracy and battery usage for mobile users. To balance precision and battery consumption, we chose reduced polling intervals for GPS updates. In the case of LAMS, this decision was made to focus on easy setup and dev speed over scalability for larger datasets (i.e., we chose to use MySQL instead of other databases like PostgreSQL).

5. Coding guidelines and standards

Naming Convention: camelCase on all classes, variables and methods for uniformity.

Comments: Each function and method includes information about what it does and what the parameters are for subsequent readability.

Commit Messages - The comitter's name is the first part of a commit, followed by a concise note detailing what was changed like this; "Josh - Integrated GPS with Map Component."

Branching Strategy : Feature branches to develop new features PR to main branch either need atleast one reviewer.

5. # Protocol for any one, or distinct ones of subsystems, modules, subroutines

The GPS subsystem, leveraging the Navigator API to fetch the location of current location of user. A Leaflet ApI integrated mapping display subsystem to display the map and mark locations. Both will talk jv with each other talking and the whole front end UI at these elements integrated to navigate seamlessly.

5. That is in To the choice of an algorithm or a programming idiom (or even a design pattern) in order to implement parts of a system functionality there

To separate the UI from the business logic, we adopted Model-View-Controller (MVC) design pattern. Navigation System: The Navigation system calculates the best paths between buildings, class-rooms with a given input. Using Shortest path Algorithm.

5. 7 Software lifecycle maintenance plans

Work is Version Controlled: using GitHub to version our work we can track changes, roll back and have multiple developers contribute.

Documentation: All modules have documentation and README is kept up to date with relevant changes and use cases.

You are responsible for bug tracking and have to log, prioritize and resolve the issues in Jira.

Updates The app will be reviewed and updated every semester to make sure the current class schedule & building info. (CS3337_SWE_SDD_Group6.csv) d...)(Change Management Plan-1)

5. # End-user, software, hardware,communications interfaces.

The User Interface The UI contains a map that you can interact with and has an integrated search bar, as well as separate pages for the login, and profile. It works in both your desktop and mobile browsers.

Hardware Interface The WEB Application Needs GPS Location Services Requires Gps Hardware

Communication Interface: As shown in the integrations diagram below, the web app uses an HTTP request to communicate with a MySQL database to retrieve classroom and building data, which is displayed in our map interface.

5. # Physical Component Orientation of Source Diagrams

/src — The source code for the backend (Java) and frontend (JavaScript/HTML).

/test: Backend and frontend components—unit tests

/WebContent (All the static resources(images /css and Js files) that are used in web app.

/database : This contains folder called sqlscript / SQL script for database setup and initialisation 3.

5. How the system should be built and/or generated (compact, link, load so forth.)

Building and running Scholar Path

Clone the GitHub repository.

Then, find the directory /src and compile the backend application with this command: mvn clean install

More of a web server (like Tomcat) based back end deployment.

Frontend: HTML, JavaScript, and CSS Files can be hosted directly

The MySQL database should be instantiated with given SQL scripts and the properties linked in (CS3337_SWE_SDD_Group6. d...)(CS3337_SWE_Software_Testing_Course_Assignments)

5.# How to build and/or generate the system's deliverables (how to compile, link, load, etc.)

- 1.** Backend Compilation: Use mvn clean install in the /src directory to compile and build the Java backend.
- 2.** Frontend Deployment: Deploy the static HTML, CSS, and JavaScript files to a web server.
- 3.** Database Setup: Use the provided SQL scripts to initialize the MySQL database, and configure the connection in the properties file.

5.10 Describe tactics such as abstracting out a generic DatabaseInterface class

To switch databases (e.g., MySQL to PostgreSQL), we abstract the data access layer through a DatabaseInterface class. Implementing this class allows the system to interact with any database by simply rewriting the class for the new database without changing the rest of the code.

For this particular section, it may become difficult to decide whether a particular policy or set of tactics should be discussed in this section, or in the System Architecture section, or in the Detailed System Design section for the appropriate component. You will have to use your own "best" judgement to decide this. There will usually be some global policies and tactics that should be discussed here, but decisions about interfaces, algorithms, and/or data structures might be more appropriately discussed in the same (sub) section as its corresponding software component in one of these other sections.

6. Detailed System Design

6.x Name of Component (Module)

6.x.1 Responsibilities

This module is responsible for integrating both the GPS system and the interactive map. Its services contain real-time location tracking and visual map rendering.

6.x.2 Constraints

Accuracy of GPS data is the limit on the module and an internet connection is needed for map updates. Even has to work between devices (mobile or desktop)

6.x.3 Composition

It is a two component module — GPS Service and the Map Display (uses Leaflet)

6.x.4 Uses/Interactions

This module communicates with the database in order to extract class room locations, and shows them on a map. Works together with user profile module to save favorite places.

6.x.5 Resources

Module functionality is responsible for such things as: interact with device GPS, memory to cache map, and server request to get some needed map-data.

6.x.6 Interface/Exports

This module supplies:

Services: GPSService for requesting the user location, MapService for rendering maps.

Statics: Navigator API and Leaflet API (for location and map fetching)

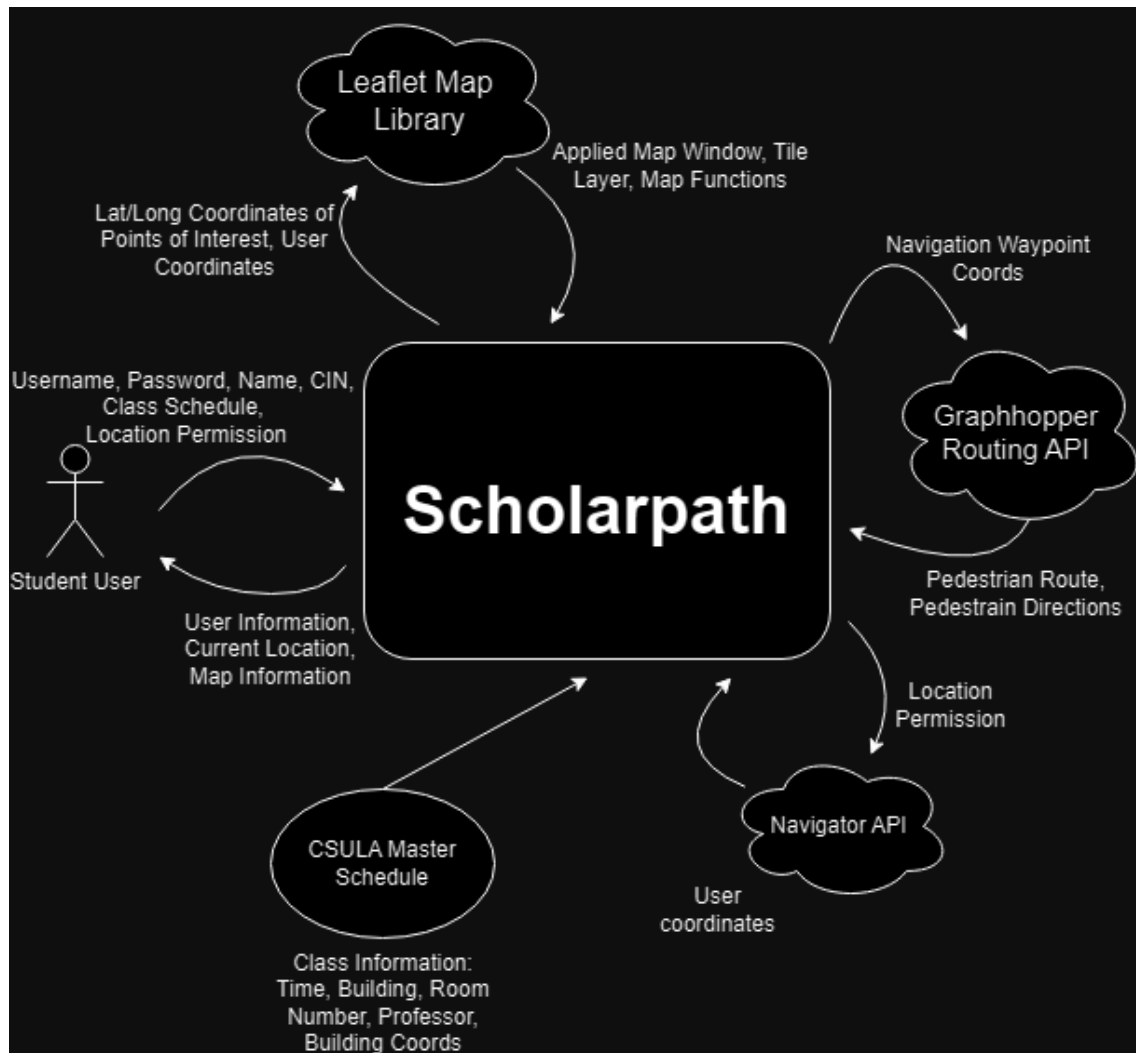
Values: coordinates (latitude, longitude) for GPS tracking, the information about building & classroom is returned from database;

Methods:

getCurrentLocation(): request the equipment for location in real time

displayMap(): Displays map with user location and destinations.

The findPath(): reserves the path to *this* classroom. All Service designed to handle maps exceptions due to gps fails or no data.



Data Flow Diagram

The product will interact with each external source, where no external sources are communicating with each other. The user will provide necessary information, the product will then check its database created from the information of the CSULA Master Schedule. Next, the product will show the allowed location to the Navigator API, with Navigator providing the user's coordinates in return. With these coordinates and those of the points of interest within the product, they are displayed by the leaflet map library by assigning the data to the appropriate centering and marker coordinates. These coordinates are also sent to the Graphhopper API in conjunction with Leaflet Routing Machine (LRM) to overlay a pedestrian route and turn-by-turn

directions on the map interface. The information within the map and profile interfaces are then returned to the user.

7. Detailed Lower level Component Design

7.x Name of Class or File (Ahin Huq)

7.x.1 Classification

This is one class in GPS subsystem to interact with location.

7. 5.2 Processing Narrative (PSPEC)

This class fetches the current GPS location of the user with help of Navigator API, then works on to modify and again displays on map view to show real time position of the user.

7.x.3 Interface Description

The following are the methods this class exposes

getCurrentLocation(): it returns the user current coordinates.

updateMap(): This will update the map display with user's location.

7.x.4 Processing Detail

7. x.4. 1 Design Class Hierarchy

This Class extends with BaseLocationService class and there may further parentize with other specific services like for indoor navigation.

7. x.4. 2 Restrictions/Limitations

Class uses location data, depends on gps accuracy and may not work indoors ×

7.x.4.3 Performance Issues

This feature of constant location updates can mean this using extra battery on cell phones. It is the balance between how often it gets updated, and when can it get us what we want which makes it important.

7.x.4.4 Design Constraints

The class need to run on both desktop and mobile, including different screen sizes and GPS.

7. x.4. 5 Operational Processing Detail

getCurrentLocation(): Uses navigator to extract the GPS data. geolocation. It is called the getCurrentPosition() and converted to map coordinates.

updateMap() : Updates the map center with new location & updates user marker on the map.

8. Database Design (Joshua Hanscom)

Include details about any databases used by the software. Include tables and descriptions.

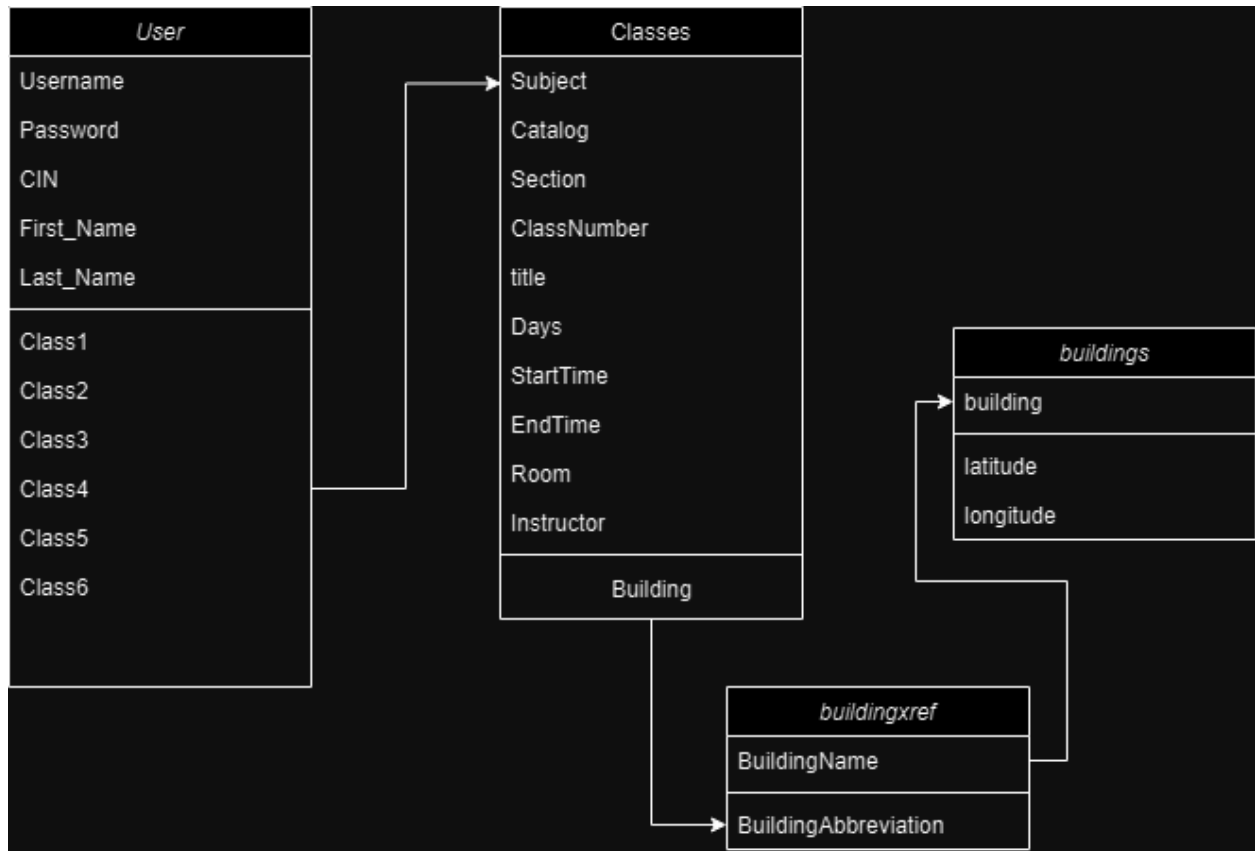
USER
Username
Password
CIN
First_Name
Last_Name
Class1
Class2
Class3
Class4
Class5
Class6

CLASSES
Subject
Catalog
Section
ClassNumber
title
Days
StartTime
EndTime
Room
Instructor
Building

BUILDINGXREF
BuildingName
BuildingAbbreviation

BUILDINGS
building
latitude
longitude

Each table represents a different table in the database. The header represents the title of the table while the cells below represent that table's fields.



Each rectangle represents a table where the top component is the table name and below it are its associated fields. Arrows are used to provide connections to specific fields in other tables.

9. User Interface

9.1 Overview of User Interface

Describe the functionality of the system from the user's perspective. Explain how the user will be able to use your system to complete all the expected features and the feedback information that will be displayed for the user. This is an overview of the UI and its use. The user manual will contain extensive detail about the actual use of the software.

9.2 UX Standards

9.3 Screen Frameworks or Images

These can be mockups or actual screenshots of the various UI screens and popups.

9.4 User Interface Flow Model

A discussion of screen objects and actions associated with those objects. This should include a flow diagram of the navigation between different pages.

10. Requirements Validation and Verification(Charlie Kaing)

Create a table that lists each of the requirements that were specified in the SRS document for this software.

For each entry in the table list which of the Component Modules and if appropriate which UI elements and/or low level components satisfies that requirement.

For each entry describe the method for testing that the requirement has been met.

Component	Testing criteria
Map	Map shows all available schedules as well as precise coordinates and direction
Database	Successfully retrieves database information as well as relevant information
Student information	Retrieves information from students CIN and verifies login information
Classroom location	Map should show the correct location classes on the map as well as directions.

11. Glossary (Ahin Huq)

Application Programming Interface: A framework of tools that facilitates the interaction between software and other programming languages

Backlog: An ordered list of tasks or new features that have to be completed within a software project.

Branching Strategy: Branching is the concept used in version control systems like git in which different features and updates are developed in separate line of codes to have a better stability.

Class hierarchy: This is the structure that defines parent-child relationships between classes in object-oriented programming.

Commit: The act of storing a change on the codebase, normally with description of what changed.

Concurrency: The execution of many operations simultaneously.

A DFD (Data Flow Diagram) is a graphical representation of how the data flows within this system, usually to show its architecture.

Epics A large body of work that can be broken down into a number different smaller tasks, or user stories in project management, particularly Scrum.

GPS (Global Positioning System): A satellite-based system used to determine the real-time location of a device.

Jira: a software tool which is helpful in issue tracking and agile project management.

Leaflet API: A javascript library for making interactive maps with js.

Model: View–Controller: A design pattern that separates an application into three main logical components: the model (data), the view (user interface), and the controller (input).

Navigator API: A web API for obtaining the geographic location of the users.

PostgreSQL: Postgres is an open-source, highly versatile database management system designed to scale and handle complex sql query efficiently.

REST API (Representational State Transfer): It is a style of architecture where communication happens between client and server using HTTP requests.

Sprint: A Sprint is a short, time-boxed period when additional specific work or features are completed and prepared for review within Agile development.

SQL: Structured Query Language: This is a standard language which has been made to easily manage and query databases.

User story: Simple one or two sentence description of a new feature, it is written so that its understandable to the user.

12. References

<List any other documents or Web addresses to which this SDD refers. These may include other SDD or SRS documents, user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location.>

Brad Appleton <brad@bradapp.net> <http://www.bradapp.net>

https://www.cs.purdue.edu/homes/cs307/ExampleDocs/DesignTemplate_Fall08.doc